

CS236 Operating Systems Lab

Spring 2025-26

Lab 12: drivers needed, apply now!

xv6 is a teaching operating system (developed at MIT) — an OS meant for teaching design and implementation concepts of an OS.

The Holy Grail for xv6

- [xv6-risc book](#)
- [xv6-risc src booklet](#)

The xv6 source tree is organized to mirror OS subsystems:

- **kernel/** → process management, memory, file system, traps
- **user/** → user programs
- **mkfs/** → file system creation tool

Each directory corresponds to a logical OS component, enabling systematic exploration and modification. **You will mostly interact with the **kernel/** and **user/** directory.**

Submission Guidelines

Create a patch with:

```
Shell
> ./patch.sh
```

Rename the generated **lab12.patch** to **<rollno>_lab12.patch** and submit via Moodle.

0. The magic of memory-mapped I/O (MMIO)

When you write any C program and try to access a memory (using some address), you likely imagine the CPU accessing the RAM to retrieve or store a value. For standard applications, this is true. But for operating systems, not all memory addresses point to RAM.

On most modern architectures, the CPU communicates with hardware devices using a technique called Memory-Mapped I/O (MMIO). For e.g. the STDIN/STDOUT from the console of xv6 is actually handled by a device known as UART. The UART hardware appears to software as a set of memory-mapped control registers. That is, some physical addresses are connected to the UART device, so that loads and stores interact with the device hardware rather than RAM.

For this task, you need to find the starting physical addresses of the following devices:

- **UART device used for console**
- **VIRTIO device used for disk**
- **e1000 device used for networking (part of this lab)**

Create a new file, `task0.txt` (in `xv6-riscv/`), and write the three addresses (in hex) on three different lines.

Hint:

- Look at `kvmmake()` in `kernel/vm.c`
- Look at Fig 3.3 in the xv6 book.

1. Phantom keystrokes

When you type a character on your keyboard, QEMU simulates a hardware interrupt. The xv6 kernel catches this interrupt and routes it to the UART driver (`xv6-riscv/kernel/uart.c`).

Currently, the flow is like this:

1. The hardware tells the CPU,
"I have a character!" (`devintr()` in `xv6-riscv/kernel/trap.c`)
2. `uartintr()` is invoked, which reads the single character using `uartgetc()` with MMIO.
3. The driver then passes the character to the OS via `consoleintr()`.

The character is then buffered in an input ring buffer, which is then read by user programs via the `read()` system call.

Your task is to intercept a specific keystroke and use it for a text replacement command before it reaches the software console.

For example, when you press the “!” key (ASCII 33), instead of inputting the exclamation mark, the kernel automatically replaces the single character with a string of characters like “echo Hello World” (or a string of your choice) into the console. To the OS, it should look exactly as if a user typed the string of characters at an incredible speed.

Hints:

- Look up the `uartintr()` function in `kernel/uart.c` and locate the function that processes incoming hardware interrupts.

2. Pulse check

xv6 has some hidden console shortcuts, like (Try this out on your qemu terminal):

- **Ctrl+P** to print the current process list
- **Ctrl+H** to act like a backspace/delete

These shortcuts are implemented in the **consoleintr** handler, which is invoked by the UART device driver.

Now try **Ctrl+M** and check the behavior. Although this shortcut is not implemented, some action seems to be happening. Try to understand the reason for the same.

Hint: Look up the ASCII table for control and printable characters.

Your task is to add a new console shortcut, **Ctrl+I**, which prints current hardware device stats. Specifically, you need to print the number of times interrupts have occurred due to the following devices — clock, uart, virtio, and e1000.

Hints:

- A global data structure `trap_stat` is provided to hold the counters (in `trap.c`)
- A function (`trapdump()`) to print the statistics is also provided (in `trap.c`)

Todo: Use the data structure for bookkeeping of the interrupts and invoke the `trapdump` function on the keyboard shortcut.

Sample Output:

```
Shell
$
c      u      v      n
153    30     34     0

$ echo hello
hello
$
c      u      v      n
495    66     39     0

$
c      u      v      n
746    72     39     0
$ echo hello > file.txt
$
c      u      v      n
2333   155    67     0
```

The output shows that timer interrupts occur frequently, UART interrupts occur when interacting with the console, and VIRTIO interrupts occur when interacting with the disk.

3. Let's get connected

So far, we have looked at the built-in UART device (which handles single-character I/O). Now, we will interact with a new device, the e1000. e1000 is an Ethernet controller that provides networking support to the xv6 OS.

Like the UART, the e1000 uses MMIO for communication with the CPU. However, instead of passing single characters back and forth, the e1000 processes entire network packets using structures called **descriptors**. Most of the device driver functionality for interacting with the e1000 device is provided in this lab.

In your lab setup, we have configured the networking so that UDP packets sent to the host machine (e.g., sl2-23 on ports 25999 and 30999) are forwarded to xv6 on ports 2000 and 2001. Assume that port numbers are unique addressable endpoints available on the same machine.

This can be tested using the nettest utility available on the host machine.

Usage: python3 nettest.py rx

You can observe that network interrupts using Ctrl+I inside qemu on the xv6 terminal —

```
Shell
$
c      u      v      n
54     22     34     0

c      u      v      n
279    24     34     4

c      u      v      n
327    26     34     6

c      u      v      n
570    28     34     14
```

Before we start sending and receiving packets, let's verify that our xv6 kernel can talk to the device.

Your task is to add another console shortcut, **Ctrl+N**, that prints the status of the e1000 device. Note that e1000 registers are already memory-mapped to physical addresses. The relative location of the status register from the start of the e1000 MMIO base address is defined by the macro E1000_STATUS.

To read the status, you will need to read the value from the location above. Use the given macros (upstatus(), speed(), duplex()) to implement the e1000_status() (in e1000.c) function and invoke it appropriately.

Sample Output:

```
Shell
$
Link up, Speed 1000Mbps, full duplex
```

4. Sending Packets over the Wire

To send a packet, the OS does not just hand the data directly to the hardware via MMIO. Instead, the e1000 uses a Transmit (TX) Ring Buffer. This is an array of "descriptors" in memory.

When the OS wants to send a packet, it places the packet's physical memory address in the next available descriptor in the tx ring, updates a specific MMIO register to tell the e1000 device — "I have a new packet to transmit!". The e1000 hardware then reads, interprets the descriptor, and sends the packet out through the NIC.

The Mechanics of the Rings:

```
C/C++
// The tx descriptor
struct tx_desc
{
    uint64 addr;        /* Address of the descriptor's data buffer */
    uint16 length;      /* Length of data to be copied from data buffer */
    uint8 cso;
    uint8 cmd;          /* Descriptor command */
    uint8 status;       /* Descriptor status */
    uint8 css;
    uint16 special;
};

#define TX_RING_SIZE 16
static struct tx_desc tx_ring[TX_RING_SIZE]; // This is the shared ring buffer
```

Before diving into the code, you need to understand how the e1000 interacts with memory. The hardware and software share a circular array (a ring) of descriptors.

Imagine a circular track with two runners: Head (the hardware) and Tail (the software).

During transmission, the OS adds newly created network packets to the Tail and advances the Tail pointer. The hardware follows behind, reading packets at the Head, sending them over the wire, and advancing the Head pointer.

Because checking bitmasks and wrapping around circular buffers can get messy, we have provided a few helpful macros you should use in your implementation:

```

C/C++
// Helpful Macros for Ring Management

// Peek to find current tail index
#define CURR_TX_IDX regs[E1000_TDT]

// Calculate the next tail index for queuing next packet in ring
#define NEXT_TX_IDX(idx, size) (((idx) + 1) % (size))

// Check if tx at index is done (returns 0 if done)
#define TX_DONE(tx_ring, idx) ((tx_ring[idx].status) & E1000_TXD_STAT_DD)

// Status value to notify packet to be sent
#define TX_SEND 0

// Command value to notify packet to be sent
#define TX_CMD_FLAGS (E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS)

```

When the xv6 network stack has a packet to send obtained via the send system call, it passes a memory buffer (buf) (a kernel virtual address) to `e1000_transmit()` in `kernel/e1000.c`. Your job is to add this buffer to the TX ring so the hardware can find and transmit it. Note that the send system call and utilities to invoke the call have been provided as part of the lab.

You should do the following for this task:

1. Read the current software tail index using `CURR_TX_IDX`.
2. Check if the hardware is done with the descriptor at the tail index using the `TX_DONE` macro on its status field. If it is not done (returns 0), the ring is full. Return -1 in that case.
3. Since the ring wraps around, the descriptor you are about to use might contain the physical address of a packet sent a long time ago. You should free the page pointed to by `addr` if it is not null, using `kfree()`.
4. Set up the descriptor (`tx_ring[idx]`) for transmission as follows:
 - a. Set `addr` to your new buf.
 - b. Set `length` to `len`.
 - c. Set `cmd` using `TX_CMD_FLAGS`
 - d. Set `status` to `TX_SEND`
5. Update the `CURR_TX_IDX` using the `NEXT_TX_IDX` macro.

Note: You have to acquire `e1000_lock` to ensure there are no concurrency issues

Testing:

Outside on the host machine, start a UDP server -

```
Shell
$ python3 nettest.py txone
tx: listening for a UDP packet
txone: OK
$
```

Inside xv6, use nettest to send a UDP packet to the server -

```
Shell
$ nettest txone
txone: sending one packet
```

You can check the packet that was sent from xv6 using tcpdump on the host machine:

```
Shell
$ tcpdump -XXnr packets.pcap
reading from file packets.pcap, link-type EN10MB (Ethernet), snapshot length
65536
04:33:24.838428 IP 10.0.2.15.2003 > 10.0.2.2.25099: UDP, length 5
    0x0000:  5255 0a00 0202 5254 0012 3456 0800 4500  RU....RT..4V..E.
    0x0010:  0021 0000 0000 6411 3ebc 0a00 020f 0a00  .!....d.>.....
    0x0020:  0202 07d3 620b 000d 0000 7478 6f6e 65    ....b.....txone
```

5. Receiving Packets over the Wire

When a packet arrives over the network, the e1000 hardware automatically copies the payload from the wire directly into memory. To manage this, it uses a Receive (RX) Ring Buffer, which is another array of descriptors.

The Mechanics of the Rings:

```
C/C++
// The rx descriptor
```

```

struct rx_desc
{
    uint64 addr;           /* Address of the descriptor's data buffer */
    uint16 length;         /* Length of data copied into data buffer */
    uint16 csum;
    uint8 status;          /* Descriptor status */
    uint8 errors;
    uint16 special;
};

#define RX_RING_SIZE 16
static struct rx_desc rx_ring[RX_RING_SIZE]; // This is the shared ring buffer

```

The receive ring works similarly to the transmit ring, but the roles are reversed.

The OS pre-fills the ring with empty memory buffers (done in `e1000_init`) and waits at the tail to read incoming packets. When the hardware receives packets from the wire, it writes the data into the empty buffers at the Head, sets the DD (Descriptor Done) bit to say "I wrote a packet here!", and advances the Head pointer.

When a packet arrives, it triggers a hardware interrupt, which eventually calls `e1000_recv()` in `kernel/e1000.c`. Your job is to extract these incoming packets and hand them off to the `xv6` network stack, implemented as the kernel function `net_rx()`.

Just like in transmission, we have provided a few helpful macros to make ring management easier:

```

C/C++
// Helpful Macros for Ring Management

// Peek to find current tail index
#define CURR_RX_IDX regs[E1000_RDT]

// Calculate the next tail index for queuing next packet in ring
#define NEXT_RX_IDX(idx, size) (((idx) + 1) % (size))

// Check if packet are available at index (returns 0 if not available)
#define RX_AVAILABLE(rx_ring, idx) ((rx_ring[idx].status) & E1000_RXD_STAT_DD)

// Status value to notify packet is processed
#define RX_CLEAR 0

```

You should do the following for this task:

1. Read the current index using `CURR_RX_IDX`. The software processes packets starting at the next index, so immediately calculate the next index using `NEXT_RX_IDX`.
2. Hardware is fast, and multiple packets might have arrived before the interrupt handler fired. Use a while loop to check if the hardware has written a packet at your current index using the `RX_AVAILABLE` macro. Keep looping until `RX_AVAILABLE` returns true.
3. Inside the loop:
 - a. Extract the packet details: the address and the length.
 - b. Pass the packet up to the OS network stack by calling `net_rx(buf, len)`
 - c. You just took a buffer away from the hardware, so you must replace it! Allocate a fresh buffer using `kalloc()`.
 - d. Update the descriptor's `addr` with your new buffer.
 - e. Reset the descriptor's status to `RX_CLEAR`.
 - f. Tell the hardware about the newly replenished buffer by updating `NEXT_RX_IDX` to the current index.
 - g. Advance the index variable using `NEXT_RX_IDX` and continue the while loop to check for more packets.

Note: Partial implementation has been provided; you need to complete the rest.

Testing:

Inside xv6 -

```
Shell
$ nettest rxone
rxone: starting
arp_rx: received an ARP packet
ip_rx: received an IP packet
rxone: got "Hello OS"
rxone: OK
$
```

Outside in the host machine -

```
Shell
$ python3 nettest.py rxone Hello OS
rxone: sending one UDP packet: 'Hello OS'
$
```

You can check the packet received at xv6 using tcpdump on the host machine:

```
Shell
$ tcpdump -XXnr packets.pcap
reading from file packets.pcap, link-type EN10MB (Ethernet), snapshot length
65536
10:43:38.195028 ARP, Request who-has 10.0.2.15 tell 10.0.2.2, length 46
    0x0000:  ffff ffff ffff 5255 0a00 0202 0806 0001  .....RU.....
    0x0010:  0800 0604 0001 5255 0a00 0202 0a00 0202  .....RU.....
    0x0020:  0000 0000 0000 0a00 020f 0000 0000 0000  .....
    0x0030:  0000 0000 0000 0000 0000 0000  .....
10:43:38.195752 ARP, Reply 10.0.2.15 is-at 52:54:00:12:34:56, length 28
    0x0000:  5255 0a00 0202 5254 0012 3456 0806 0001  RU....RT..4V....
    0x0010:  0800 0604 0002 5254 0012 3456 0a00 020f  .....RT..4V....
    0x0020:  5255 0a00 0202 0a00 0202  .....RU.....
10:43:38.195884 IP 10.0.2.2.38575 > 10.0.2.15.2000: UDP, length 8
    0x0000:  5254 0012 3456 5255 0a00 0202 0800 4500  RT..4VRU.....E.
    0x0010:  0024 0000 0000 4011 62b9 0a00 0202 0a00  .$....@.b.....
    0x0020:  020f 96af 07d0 0010 d5f8 4865 6c6c 6f20  .....Hello.
    0x0030:  4f53 0000 0000 0000 0000 0000  OS.....
```

Bonus Testing:

For testing both sending and receiving, you can do the following outside in the host:

```
Shell
$ python3 nettest.py grade
txone: OK
rxone: sending one UDP packet
^C
[INFO] Interrupted by user. Exiting cleanly.
$
```

Inside xv6:

```
Shell
$ $ nettest grade
txone: sending one packet
arp_rx: received an ARP packet
ip_rx: received an IP packet
```

```
ping0: starting
ping0: OK
ping1: starting
ping1: OK
ping2: starting
ping2: OK
ping3: starting
ping3: OK
dns: starting
DNS arecord for www.cse.iitb.ac.in. is 10.129.3.3
dns: OK
free: OK
$
```